

VarejOnline

Integração VarejOnline → Suri

Guia de Arquitetura e Implementação

Visão Geral

A integração entre **VarejOnline** (ERP) e **Suri** sincroniza automaticamente catálogo, estoque, preços e lojas do ERP para a Loja Suri, e devolve ao ERP os pedidos fechados na Suri.

A solução foi desenvolvida como **plugin desacoplado**, permitindo integrar novos clientes apenas por configuração, sem mudanças no core da aplicação. O plugin roda sobre a Loja Suri nativa (provider `Suri`) — não substitui o provider de loja.

Arquivos: `ChatbotMaker.BDK/Plugins/SuriPlugins/Plugins/Varejonline/` **Tela do Portal:** `Loja → Plugins → VarejOnline`

Como Conectar o VarejOnline na Suri

Dado obrigatório

Token de integração Token de acesso da API, solicitado ao suporte técnico da VarejOnline (`suportetecnico@varejonline.com.br`). É a única credencial necessária.

Configurações avançadas (recolhidas por padrão)

Código da operação de venda — default `1` Enviado em cada item do pedido (`itens[].operacao`). A API exige o campo, o código varia por base e **não existe endpoint que liste as operações**. `1` = Venda na maioria das bases; confirmar com o suporte da VarejOnline.

Sincronizar apenas produtos marcados para e-commerce — default desligado Aplica o filtro `somenteEcommerce` da API. Depende de o campo estar configurado no ERP; se nenhum produto aparecer no catálogo após a sincronização, é este o motivo.

 Nenhum outro parâmetro é pedido ao operador. Tudo o que a API consegue informar — níveis de categoria, entidades, tabelas de preço — é descoberto automaticamente.

Autenticação

Diferente de outros ERPs, o VarejOnline **não usa Bearer Token**. A autenticação é por **token fixo na query string** de cada requisição:

```
1 GET https://integrador.varejonline.com.br/apps/api/produtos?token=<token>&inicio=0&quantidade=200
```

Características:

- Token é permanente — não expira, não precisa de refresh
- Não há fluxo de login nem renovação a gerenciar
- O token é validado no início de cada sincronização via `GET /dados-login`; se falhar, o sync para antes de gravar qualquer coisa
- A URL montada carrega a credencial, portanto **nunca é logada** — mensagens de erro usam apenas o path

Existe também OAuth 2.0 via DevCenter da VarejOnline. **Foi descartado deliberadamente:** publicar um app na store deles exporia a Suri na vitrine para operadores que não são nossos clientes. O ganho seria apenas não pedir a chave ao cliente.

Arquitetura da Integração

1. Plugin de Integração (`VarejonlinePlugin`)

Camada central de regras de negócio. Responsável por:

- Orquestrar o fluxo de sincronização
- Gerenciar estado e progresso
- Montar e persistir o modelo da Suri
- Enviar pedidos ao ERP
- Validar estoque em tempo real no fechamento do pedido

2. Client HTTP (`VarejonlineClient`)

Camada de infraestrutura. Responsável por:

- Comunicação com a API do VarejOnline
- Paginação automática (`inicio` / `quantidade`)
- Retry com backoff exponencial em falhas transitórias (429 e 5xx)
- Conversão do formato de data da API (`dd-MM-yyyy HH:mm:ss`)

- Controle de limite de requisições

3. Controle de Taxa (`VarejonlineRateLimiter`)

A VarejOnline limita **30 requisições a cada 2 segundos por IP**, com bloqueio de até 3 minutos ao estourar. Como todos os pods saem pelo mesmo NAT, o limite é compartilhado entre todos os chatbots — por isso o controle é **estático, por processo**, e não por instância de client. Opera com folga proposital (25 em vez de 30).

4. Jobs

- `VarejonlineSyncJob` — sincronização diária (06:00 UTC), sempre reagenda a próxima execução mesmo em caso de falha
- `VarejonlinePlaceOrderJob` — envio do pedido ao ERP com retry durável (backoff de 1 a 16 minutos, até 6 tentativas)

5. Camada de Persistência (Suri Repositories)

Persiste lojas, categorias e produtos no modelo interno da Suri, mapeando identificadores externos via `ProviderId`.

Mapeamento de Conceitos (VarejOnline → Suri)

VarejOnline	Suri	Descrição
Entidade	Seller / Store	Unidade, filial ou ponto de retirada
Nível de categoria	—	Define qual classificação vira a árvore do catálogo
Categoria (nível principal)	Category	Agrupamento de produtos
Produto base de grade	Product	Item comercial com variações
Produto simples	Product (1 dimension)	Item comercial sem variações
Variação da grade	Dimension	SKU vendável

valorAtributos	Dimension.Dimensions	Atributos da grade (ex.: COR = BORDÔ)
Id do produto	ProviderId	Identificador externo — é o que o pedido exige
codigoSKU	Sku	Código legível do produto
Tabela de preço	Dimension.Prices[idTabela]	Preço por tabela
Saldo líquido por entidade	Dimension.Stocks[idEntidade]	Estoque por loja
Terceiro	Customer	Cliente final
Pedido	Order	Pedido realizado

⚠ O modelo de produto tem TRÊS formatos, não dois

```
1 | base de grade → mercadoriaBase: true, e existe variação apontando para ele
```

variação → idProdutoBaseGrade preenchido, com valorAtributos

simples → mercadoriaBase: true, e ninguém aponta para ele

Um produto "simples" é **indistinguível** de uma "base de grade" olhando o item isoladamente — só se sabe depois de varrer o conjunto inteiro. Por isso o catálogo é carregado completo antes de qualquer gravação.

🔄 Fluxo de Sincronização

1/5 — Validação de credenciais

GET /dados-login . Falha aqui interrompe o processo antes de gravar qualquer dado.

2/5 — Lojas (Entidades)

GET /entidades → cria/atualiza um Seller por entidade ativa.

📌 **Mapeamento:** Entidade → Seller (Store)

3/5 — Carregamento do catálogo

Carrega em memória, numa única passada: níveis de categoria, categorias, produtos (com agrupamento base/variação/simples), tabelas de preço, preços por produto e saldo líquido por entidade.

4/5 — Categorias

Cria/atualiza as categorias do **nível principal** da base.

5/5 — Produtos

Monta cada produto completo — variações, preços e estoque de todas as entidades já resolvidos — e grava **uma única vez por produto**, somente quando algo mudou.

Ao final, se o chatbot usa Azure AI Search, a reindexação é disparada automaticamente.

Lojas e Pontos de Retirada

Cada **entidade ativa** do VarejOnline vira uma **loja** na Suri. É esse vínculo que permite estoque por loja e envio do pedido para a filial certa.

 A API não informa o endereço das entidades

`GET /entidades` devolve apenas nome e flags. O único endereço disponível na API é o do **terceiro da empresa** — e todas as entidades compartilham o mesmo CNPJ, então ele não distingue ponto de retirada.

Divisão de responsabilidade:

Campo	Origem
<code>ProviderId</code> , <code>Name</code>	Sincronização — atualizado a cada rodada
Endereço, CNPJ, tabelas de preço, pontos de retirada	Cadastro manual no Portal (<code>Catálogo</code> → <code>Lojas</code>)

A sincronização **nunca sobrescreve** os campos manuais. Exceção única: na **criação** da loja da entidade principal, o endereço do terceiro da empresa é aproveitado — nunca replicado para as demais, porque colocar todos os pontos no mesmo lugar quebraria a ordenação por proximidade.

 Sem endereço a loja existe e recebe pedido, mas **não aparece como opção de retirada** — a busca de lojas próximas e a política de frete dependem do CEP.

Categorias

O VarejOnline classifica cada produto em **todos os níveis ao mesmo tempo** — um produto tem simultaneamente GRUPO, MATERIAL, GÊNERO, COLEÇÃO, CATEGORIA e TRIBUTAÇÃO. A Loja Suri precisa de **uma** árvore.

A escolha não é configurada na Suri: o plugin lê `GET /niveis-categoria-produto` e usa o **nível ativo de menor ordem** — o mais abrangente, conforme a configuração da própria base do cliente.

```
1 | nivel=1 GRUPO      nivel=4 COLEÇÃO
```

```
nivel=2 MATERIAL    nivel=5 CATEGORIA
```

```
nivel=3 GÊNERO      nivel=6 TRIBUTAÇÃO
```

O casamento é sempre **por id**, nunca por nome: o campo `nivel` é numérico no endpoint de categorias e nominal dentro do produto.

Fluxo de Pedido (Checkout → VarejOnline)

Etapas

1. **Identificação da loja** — resolve a entidade que fatura o pedido
2. **Validação ou criação do cliente** — busca o terceiro por CPF/CNPJ; cria se não existir
3. **Verificação de duplicidade** — consulta por `numeroPedidoCliente` antes de criar
4. **Mapeamento dos itens** — `ProviderId` → id do produto no ERP
5. **Envio do pedido** — `POST /pedidos`

Campos enviados

```
1 | entidade.id      = loja do pedido
```

```
terceiro.id        = cliente no ERP
```

```
numeroPedidoCliente = id do pedido na Suri (chave de idempotência)
```

```
origem             = ECOMMERCE
```

`tipo` = `CLICK_COLLECT` (retirada) ou `NORMAL` (entrega)

`valorFrete` = valor do frete

`enderecoEntrega` = apenas quando não for retirada

`itens[].operacao` = código configurado no plugin

📌 Regras importantes

- **Cliente é criado automaticamente** caso não exista; CPF/CNPJ é a chave única
- **Pedido nunca duplica:** o job consulta por `numeroPedidoCliente` antes de criar, então uma tentativa depois de sucesso parcial não gera pedido novo
- **Retirada e entrega usam a mesma entidade** — a do pedido. A API não tem conceito de entidade de faturamento separada
- **Pedido sem CPF/CNPJ é recusado** — o ERP exige terceiro identificado
- **Plano de pagamento não é enviado** — o campo é opcional na API, então a base usa o padrão dela

⚠️ Envio com retry durável

O hook de pedido da plataforma roda em *fire-and-forget*: uma exceção viraria apenas log e o pedido nunca chegaria ao ERP. Por isso o envio vai para um **Job com retry** (backoff de 1 a 16 min, até 6 tentativas).

No checkout web, onde não há fila disponível, o envio é feito inline — sem retry, mas sem perder o pedido.

📦 Validação de Estoque em Tempo Real

Antes de fechar o pedido, o plugin consulta o **saldo líquido** da loja escolhida direto no ERP e bloqueia itens indisponíveis. Isso evita vender o que o PDV já baixou entre a última sincronização e o checkout.

```
1 | disponível = saldoAtual - quantidadeReservada
```

Estoque em trânsito não entra: ainda não chegou na loja.

🔄 Estratégia de Sincronização

A integração **carrega o catálogo completo** a cada execução e grava apenas o que mudou.

Por que não sincronização incremental por data: a classificação entre produto base, variação e produto simples só é possível olhando o conjunto completo. Um filtro por data traria variações

sem os pais e produtos base sem os filhos, produzindo um catálogo inconsistente.

Como o custo é controlado:

- 📄 Cada produto é montado inteiro em memória — variações, preços e estoque de todas as entidades — e gravado **uma única vez**
- 🔍 Antes de gravar, o produto é comparado com o que já está salvo; **sem mudança, sem escrita**
- 📄 Paginação automática (200 registros por requisição)
- 🔄 Retry com backoff em falhas transitórias
- 🚦 Controle de taxa compartilhado entre todos os chatbots

📌 O gargalo desta integração **não é a API do VarejOnline** — é escrita no banco. A estratégia acima é o que mantém o custo baixo: carga inicial grava tudo, rodadas seguintes gravam perto de nada.

🧩 Características Técnicas

- Arquitetura baseada em plugin isolado
 - Comunicação via REST API
 - Autenticação por token fixo em query string
 - Paginação automática
 - Controle de taxa estático por processo (limite é por IP)
 - Detecção de mudança para evitar escrita desnecessária
 - Envio de pedido com retry durável e idempotência
 - Reindexação automática no Azure AI Search
 - Logging de erros por etapa, com progresso visível no Portal
-

🚀 Como Integrar um Novo Cliente VarejOnline

1. Cadastro na Suri

- Informar o **token de integração**
- (Opcional) ajustar o código da operação de venda, se o suporte da VarejOnline indicar outro

2. Execução inicial

Na primeira sincronização são importados: lojas, categorias, produtos, variações, preços e estoque.

3. Cadastro dos endereços das lojas ⚠️

Passo manual e obrigatório. Em `Catálogo → Lojas`, informar o endereço de cada loja criada pela sincronização. Sem isso a loja não aparece como opção de retirada.

A loja da entidade principal já vem com endereço pré-preenchido.

4. Operação contínua

Job agendado diariamente. A cada execução o catálogo é recarregado e apenas o que mudou é gravado.

⚠️ Pontos de Atenção

- O primeiro sync é mais pesado: grava o catálogo inteiro
- Endereço de loja é responsabilidade do operador — a API não fornece
- Se nenhum produto aparecer no catálogo, verificar a opção "apenas produtos marcados para e-commerce": ela depende de configuração no ERP
- O código da operação de venda varia por base e não pode ser descoberto pela API
- Dependência direta da disponibilidade da API do VarejOnline
- O limite de requisições é **por IP e compartilhado** entre todos os clientes VarejOnline da plataforma

📌 Resultado Final

A integração permite que a Suri atue como camada omnichannel do VarejOnline, garantindo:

- Catálogo centralizado e sincronizado, com grade de variações
- Estoque por ponto de retirada, validado em tempo real no checkout
- Preços consistentes, incluindo tabelas de preço
- Integração completa de pedidos, com retirada e entrega

📖 Documentação Oficial de Referência (VarejOnline)

- **Wiki da API:** [🔗 Home](#)
- **Roteiro de integração com e-commerce:** [🔗 Roteiro de integração com Ecom](#)
- **Suporte técnico:** suportetecnico@varejonline.com.br